Lab 1 --- Denoising Diffusion Probabilistic Models (DDPM) Report

112550200 Wu-Qian Zheng

Task 1 — Swiss Roll

1. Implementation of TODOs

1.1 ddpm.py TODO 1

- Retrieve the cumulative product $\bar{\alpha}_t$ at timestep t and reshape/broadcast it to match $\mathbf{x_0}$ (correct device and shape).
- Compute $\sqrt{\bar{\alpha}_t}$ — the coefficient for the clean sample $\mathbf{x_0}$.
- Compute $\sqrt{(1 - \bar{\alpha}_t)}$ — the coefficient for Gaussian noise $\boldsymbol{\varepsilon}$.
- Form the noised sample using the closed form: $\mathbf{x_t} = \sqrt{\bar{\alpha}_t} \cdot \mathbf{x_0} + \sqrt{(1 - \bar{\alpha}_t)} \cdot \boldsymbol{\varepsilon}$.

```
######## TODO ########
# DO NOT change the code outside this part.
alphas_prod_t = extract(self.var_scheduler.alphas_cumprod, t, x0)
alphas_prod_t_sqrt = alphas_prod_t.sqrt()
one_minus_alphas_prod_t_sqrt = (1 - alphas_prod_t).sqrt()
xt = alphas_prod_t_sqrt * x0 + one_minus_alphas_prod_t_sqrt * noise
#######################
```

1.2 ddpm.py TODO 2

- Predict the noise $\hat{\boldsymbol{\varepsilon}}$ from the network at (xt ,t): $\hat{\boldsymbol{\varepsilon}} = \boldsymbol{\varepsilon}$_{\theta}(x_t, t).
- Compute the **posterior mean**
- Compute the **posterior variance**
- **Reverse step:**

  if *t>0*: sample z~N(0,I) and set xt−1 =μ+σt z;

  if *t=0*: set *xt−1=μx_{t-1}* (no added noise).

```python
# 1. predict noise
eps_pred = self.network(xt, t)

# 2. Posterior mean
# μ(x_t, t) = (1/√α_t) * (x_t - (β_t/√(1-ᾱ_t)) * ε_pred)
mean = (1 / alpha_t.sqrt()) * (xt - eps_factor * eps_pred)

# 3. Posterior variance
# σ²(t) = β_t * (1-ᾱ_{t-1})/(1-ᾱ_t)
variance = beta_t * (1 - alpha_bar_t_prev) / (1 - alpha_bar_t)

# 4. Reverse step
if t.item() > 0:
    # Sample z ~ N(0, I) for t > 0
    z = torch.randn_like(xt)
    x_t_prev = mean + variance.sqrt() * z
else:
    # No noise for t = 0
    x_t_prev = mean
```

1.3 ddpm.py TODO 3

- Iterate timesteps **from T−1 down to 0** (reverse order).
- At each timestep **t**, perform **one reverse denoising step**: update the current sample with **p_sample** to get the next x_{t-1} .
- After the loop finishes (i.e., after t=0), the current sample is the **final denoised estimate**; set $\mathbf{x0_{pred} = x_t}$.

```python
for t in reversed(range(0, self.var_scheduler.num_train_timesteps)):
    xt = self.p_sample(xt, t)

x0_pred = xt
```

1.4 ddpm.py TODO 4

- Sample ground-truth noise $\varepsilon \sim N(0, I)$ with the same shape as x_0.
- Use q_sample(x_0, t, ε) to construct the noised input x_t.
- Feed (x_t, t) to the network to predict noise $\hat{\varepsilon}$.
- Compute the training objective as MSE($\hat{\varepsilon}$, ε) — i.e., match the predicted noise to the ground-truth noise

```python
# 2) get GT noise, and use q_sample to get x_t
eps = torch.randn_like(x0)
x_t = self.q_sample(x0, t, eps)

# 3) predict noise
eps_pred = self.network(x_t, t)

# 4) MSE loss (eps, eps_pred)
loss = F.mse_loss(eps_pred, eps)
```

1.5 network.py TODO 1

- Save the input/output sizes and the number of timesteps for time conditioning.
- Create a list to hold layers and set prev_dim = dim_in to track the current width.
- For each hidden size in dim_hids:
- add a TimeLinear(prev_dim → dim_hid, num_timesteps) layer (a linear layer that injects the timestep so the network knows which diffusion step it's processing),
- then add ReLU for nonlinearity,
- update prev_dim = dim_hid.
- Append a final Linear(prev_dim → dim_out) to produce the output.
- Wrap everything in a ModuleList so the forward pass can iterate through the layers in order.

```python
######## TODO ########
# DO NOT change the code outside this part.

self.dim_in = dim_in
self.dim_out = dim_out
self.num_timesteps = num_timesteps
layers = []
prev_dim = dim_in
for dim_hid in dim_hids:
    layers.append(TimeLinear(prev_dim, dim_hid, num_timesteps))
    layers.append(nn.ReLU())
    prev_dim = dim_hid
layers.append(nn.Linear(prev_dim, dim_out))
self.layers = nn.ModuleList(layers)
######################
```

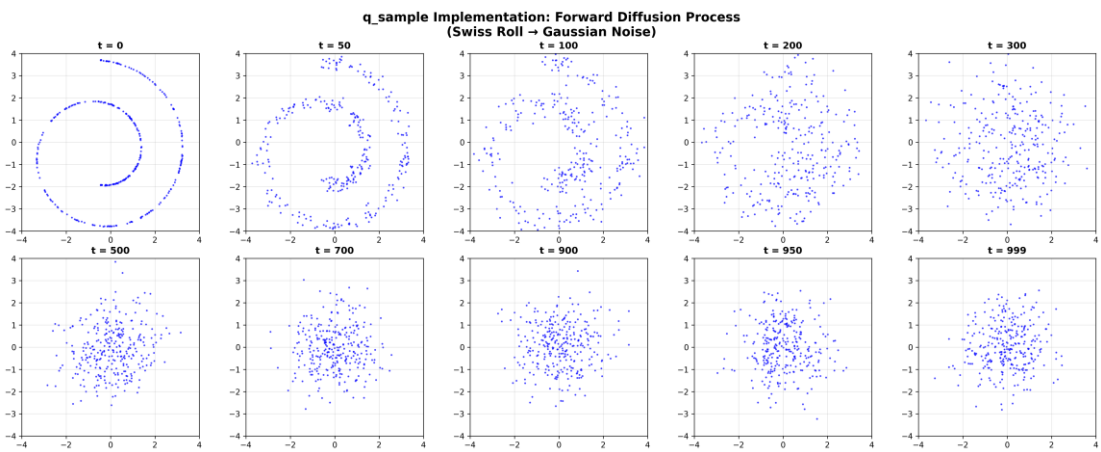
1.6 network.py TODO 2

- Iterate through the network's layers in order.
- If the layer is **TimeLinear**, call it with **(x, t)** so the timestep conditioning is injected.
- Otherwise (e.g., **Linear**, **ReLU**), call it with **x** only.
- Update **x** after each layer so it flows through the stack.
- Net effect: only time-aware layers consume **t**, while standard layers operate normally.

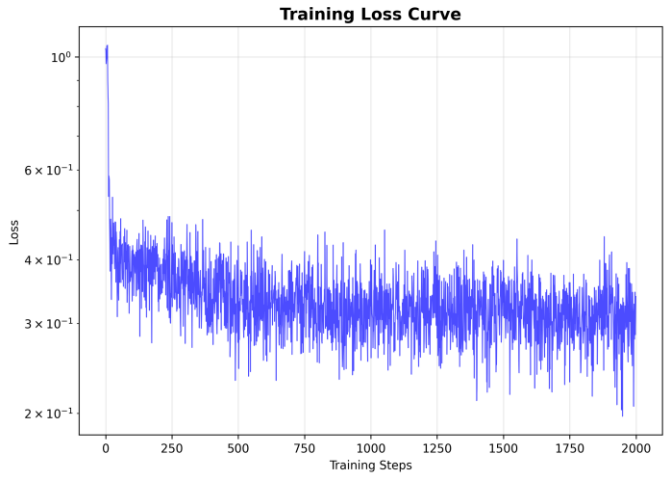
```
######## TODO ########
# DO NOT change the code outside this part.
for layer in self.layers:
    if isinstance(layer, TimeLinear):
        x = layer(x, t)
    else:
        x = layer(x)
######################
```

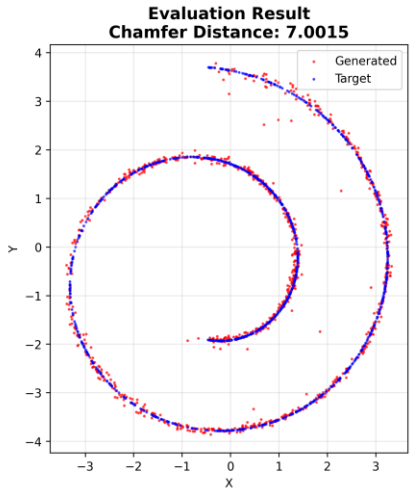
2. Results

2.1 implementation of q_sample



2.2 loss curve and evaluation result





Task 2 — image diffusion

3. Implementation of TODOs

3.1 model.py TODO 1

- Read the batch size *BB*B from x0 .
- Uniformly sample timesteps t for each example on the same device as x0 .
- Add noise to x0 at those timesteps to obtain the noised inputs xt ; also keep the injected noise ϵ.
- Predict the clean sample x^0 with the network using (xt ,t); if a class label is provided, pass it for conditional prediction.
- Train with reconstruction loss: compute MSE(x^0 ,x0) to make the model recover the clean data from the noised input.

```
B = x0.shape[0]
t = self.var_scheduler.uniform_sample_t(B, x0.device)
x_t, eps = self.var_scheduler.add_noise(x0, t, eps=noise)
x0_pred = self.network(x_t, t, class_label) if class_label is not None else self.network(x_t, t)
loss = F.mse_loss(x0_pred, x0)
```

3.2 model.py TODO 2

- Read the batch size B from x0 .
- Uniformly sample a timestep t for each item (on the same device).
- Add noise to x0 at those timesteps to get xt (and keep ϵ).
- Run the network on (xt ,t) (optionally with class_label) to get the predicted posterior meanμ^ .
- Set tprev=max(t−1,0) to access quantities at t−1.
- Gather scheduler values: αt , $\bar{\alpha}_t$, $\bar{\alpha}_{t-1}$, and β_t
- Compute the ground-truth posterior mean
- Train the"mean"predictor with MSE: loss = MSE(μ^ ,μtrue)

```
B = x0.shape[0]
t = self.var_scheduler.uniform_sample_t(B, x0.device)  # (B,)
x_t, eps = self.var_scheduler.add_noise(x0, t, eps=noise)
mean_pred = self.network(x_t, t, class_label) if class_label is not None else self.network(x_t, t)
t_prev = (t - 1).clamp(min=0)
alpha_t = self.var_scheduler._get_teeth(self.var_scheduler.alphas, t)
alpha_bar_t = self.var_scheduler._get_teeth(self.var_scheduler.alphas_cumprod, t)
alpha_bar_t_prev = self.var_scheduler._get_teeth(self.var_scheduler.alphas_cumprod, t_prev)
beta_t = self.var_scheduler._get_teeth(self.var_scheduler.betas, t)
mean_true = (alpha_bar_t_prev.sqrt() * beta_t * x0 + alpha_t.sqrt() *
             (1 - alpha_bar_t_prev) * x_t) / (1 - alpha_bar_t)
loss = F.mse_loss(mean_pred, mean_true)
```

3.3 scheduler.py TODO 1

- Set a small offset s = 0.008 (from Nichol & Dhariwal) to avoid degeneracy at the endpoints.
- Let T = num_train_timesteps and build a normalized grid t ∈ [0,1] with T+1 points (so it includes both 0 and 1).
- Compute the cumulative schedule ᾱ_t with the cosine rule:
  ᾱ_t = cos²(((t + s) / (1 + s)) · π/2).
- Normalize so ᾱ_0 = 1 (divide by the first element; clamp to avoid divide-by-zero).
- Convert cumulative terms to per-step noise levels:
  α_t = ᾱ_t / ᾱ_{t−1}, hence β_t = 1 − α_t = 1 − (ᾱ_t / ᾱ_{t−1}) (implemented as alpha_bar[1:]/alpha_bar[:-1]).
- Clamp β_t to [1e−8, 0.999] for numerical stability (keep variance positive and < 1).
- Cast β_t to float32 for training.

```python
import math
s = 0.008
T = int(num_train_timesteps)
t = torch.linspace(0, T, steps=T + 1, dtype=torch.float64) / T
alpha_bar = torch.cos(((t + s) / (1.0 + s)) * (math.pi / 2.0)) ** 2
alpha_bar = alpha_bar / alpha_bar[0].clamp(min=1e-12)
betas = 1.0 - (alpha_bar[1:] / alpha_bar[:-1])
betas = betas.clamp(1e-8, 0.999)
betas = betas.to(dtype=torch.float32)
```

3.4 scheduler.py TODO 2

- Fetch the scheduler scalars at step t on the correct device/dtype: β_t, α_t, $\bar{\alpha}_t$, and $\bar{\alpha}_{(t-1)}$ (use 1 when *t=0t=0*t=0).
- Compute the DDPM posterior mean
- Compute the posterior variance
- Reverse update: if t>0, sample z~N(0,I) and set

  $x_{t-1} = \mu\theta + \tilde{\beta}_t \; z$

```python
######## TODO ########
# 1. Extract beta_t, alpha_t, and alpha_bar_t from the scheduler.
# 2. Compute the predicted mean μ_θ(x_t, t) = 1/√α_t * (x_t - (β_t/√(1-ᾱ_t)) * ε̂_θ).
# 3. Compute the posterior variance \tilde{β}_t = ((1-ᾱ_{t-1})/(1-ᾱ_t)) * β_t.
# 4. Add Gaussian noise scaled by √(\tilde{β}_t) unless t == 0.
# 5. Return the final sample at t-1.
device = x_t.device
dtype = x_t.dtype
beta_t = self.betas[t].to(device=device, dtype=dtype)
alpha_t = self.alphas[t].to(device=device, dtype=dtype)
alpha_bar_t = self.alphas_cumprod[t].to(device=device, dtype=dtype)
alpha_bar_prev = (self.alphas_cumprod[t - 1].to(device=device, dtype=dtype)
                  if t > 0 else torch.tensor(1.0, device=device, dtype=dtype))
mean = (x_t - (beta_t / torch.sqrt(1.0 - alpha_bar_t)) * eps_theta) / torch.sqrt(alpha_t)
posterior_var = ((1.0 - alpha_bar_prev) / (1.0 - alpha_bar_t)) * beta_t
posterior_var = torch.clamp(posterior_var, min=1e-20)
if t > 0:
    noise = torch.randn_like(x_t)
    sample_prev = mean + torch.sqrt(posterior_var) * noise
else:
    sample_prev = mean
#######################
```

3.5 scheduler.py TODO 3

- Pack the current step as a tensor and gather the scheduler scalars at t: βt , αt , and $\bar{\alpha}t$.

- From your x0 predictor, recover the noise estimate
- Compute the posterior mean
- If t>0: set t_{prev} =t-1, fetch $\bar{\alpha}_{t-1}$, compute the posterior variance, and update x_{t-1}.

  If t=0: no noise is added; return $x_{t-1}=\mu\theta$.

```
######## TODO ########

t_tensor = torch.tensor([t], device=x_t.device)
beta_t = self._get_teeth(self.betas, t_tensor)
alpha_t = self._get_teeth(self.alphas, t_tensor)
alpha_bar_t = self._get_teeth(self.alphas_cumprod, t_tensor)
eps_pred = (x_t - alpha_bar_t.sqrt() * x0_pred) / (1 - alpha_bar_t).sqrt()
mean = (1 / alpha_t.sqrt()) * (x_t - (beta_t / (1 - alpha_bar_t).sqrt()) * eps_pred)

if t > 0:
    t_prev = t - 1
    t_prev_tensor = torch.tensor([t_prev], device=x_t.device)
    alpha_bar_t_prev = self._get_teeth(self.alphas_cumprod, t_prev_tensor)
    variance = beta_t * (1 - alpha_bar_t_prev) / (1 - alpha_bar_t)
    noise = torch.randn_like(x_t)
    sample_prev = mean + variance.sqrt() * noise
else:
    sample_prev = mean

#######################
```

3.6 scheduler.py TODO 4

- If t > 0:

  Pack t on the right device and gather scheduler values: β_t and $\bar{\alpha}_t$.

  Set t_{prev} = t − 1 and gather $\bar{\alpha}$_{t−1}.

  Compute the posterior variance

  Sample Gaussian noise z~N(0,I) and update x_{t-1}
- Else (t = 0):No noise added; set x_{t-1} = $mean_{\theta}$.

```python
######## TODO ########

if t > 0:
    t_tensor = torch.tensor([t], device=x_t.device)
    beta_t = self._get_teeth(self.betas, t_tensor)
    alpha_bar_t = self._get_teeth(self.alphas_cumprod, t_tensor)
    t_prev = t - 1
    t_prev_tensor = torch.tensor([t_prev], device=x_t.device)
    alpha_bar_t_prev = self._get_teeth(self.alphas_cumprod, t_prev_tensor)
    variance = beta_t * (1 - alpha_bar_t_prev) / (1 - alpha_bar_t)
    noise = torch.randn_like(x_t)
    sample_prev = mean_theta + variance.sqrt() * noise
else:
    sample_prev = mean_theta

######################
```

3.7 scheduler.py TODO 5

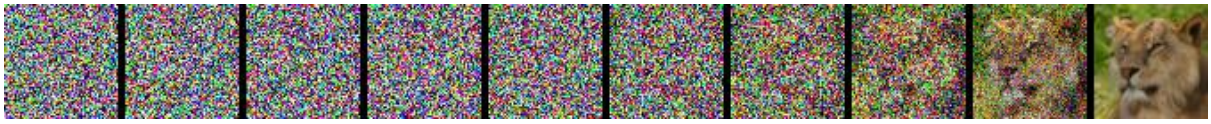
- Retrieve the cumulative product $\bar{\alpha}_t$ at timestep t from alphas_cumprod (reshaped/broadcast to match x_0).
- Form the forward noised sample using the closed form $x_t = \sqrt{\bar{\alpha}_t} \cdot x_0 + \sqrt{(1 - \bar{\alpha}_t)} \cdot \varepsilon$, with $\varepsilon \sim \mathcal{N}(0, I)$.

```python
######## TODO ########
# DO NOT change the code outside this part.
# Assignment 1. Implement the DDPM forward step.
alpha_bar_t = self._get_teeth(self.alphas_cumprod, t)
x_t = alpha_bar_t.sqrt() * x_0 + (1 - alpha_bar_t).sqrt() * eps
########################
```

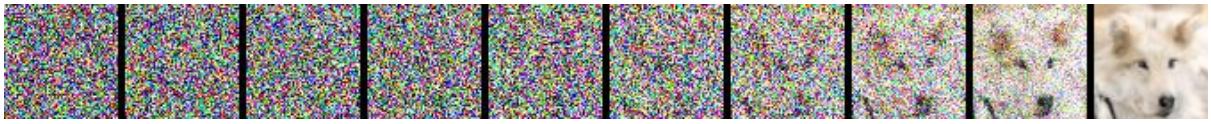
4. trajectory fig (step 99800)

4.1 trajectory fig (step 99800)

(quad_noise)



(linear_noise)



(cosine_noise)

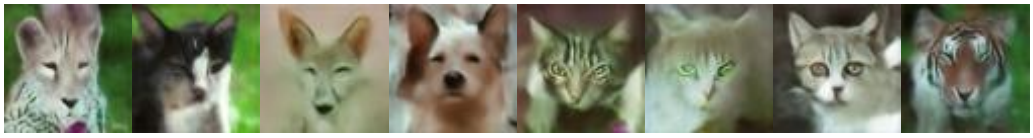
4.2 compare three beta schedulers

- Linear schedule
  - Mid-trajectory looks most chaotic; denoising progress is uneven.
  - Final image is recognizable but soft, with muddier textures and background color noise.
  - Baseline quality; weakest detail recovery among the three.
- Quadratic (quad) schedule
  - Early steps retain more structure; later steps denoise aggressively.
  - Final image is sharper than linear with clearer edges and fur strands.
  - Transition can feel a bit abrupt in late steps; overall solid but not best.
- Cosine schedule
  - Smoothest progression; structure emerges earlier and strengthens steadily.
  - Best final fidelity: cleaner background, more natural textures, and crisp details.
  - Most stable across steps; typically the preferred default.

Verdict for your results: cosine > quad > linear in final clarity, texture fidelity, and stability.

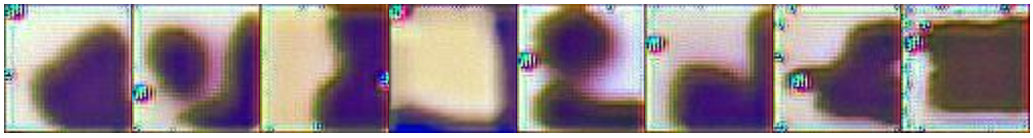
5. results of different predictors

5.1 samples per predictor



(linear_x0)

(linear_noise)



(mean_noise)

5.2 discuss

6. FID

6.1 different combination and its performance

(linear_mean)

(cosine_noise)

(linear_noise)

(quad_noise)

(linear_x0)

(cosine_x0)

6.2 Best FID and reason

In this experiment, the best result was achieved with x_0 combined with the cosine schedule, yielding an FID of 1.8989. I believe this performance comes from the strong compatibility between their characteristics. The x_0 predictor attempts to directly reconstruct the clean image at each step rather than inferring it indirectly through noise. This design places greater emphasis on fine detail reconstruction in the later steps, resulting in images with sharper contours and more natural textures. Meanwhile, the cosine schedule preserves more of the original information in the early stages and gradually accelerates denoising in the later stages, making the process both stable and progressive while avoiding premature

information loss or oversmoothing. Together, this synergy allows the model to maintain structural integrity while refining fine details in the final steps.